

MODULE 7- COLLECTIONS,FUNCTIONS AND MODULES

ASCENDING LIST

Q1. Understanding how to create and access elements in a list.

Understanding how to create and access elements in a list is fundamental to programming in many languages, including Python, JavaScript, and others. Here's a breakdown of the concepts and examples, primarily focusing on Python, which is widely used for its clarity and versatility:

1. Creating a List:

- . Syntax:** Lists are created using square brackets [], with elements separated by commas.
- . Examples:**

```
# Empty list empty_list = []
```

```
# List of integers
```

```
numbers = [1, 2, 3, 4, 5]
```

```
# List of strings
```

```
fruits = ["apple", "banana", "cherry"]
```

```
# List of mixed data types
```

```
mixed_list = [1, "hello", 3.14, True]
```

```
...
```

2. Accessing Elements:

- **Indexing:** List elements are accessed using their index, which starts from 0 for the first element.
- **Syntax:** list_name[index]
- **Examples:**

Python

```
fruits = ["apple", "banana", "cherry"]
```

```
print(fruits[0]) # Output: apple
```

`print(fruits[1])` # Output: banana

`print(fruits[2])` # Output: cherry

- **Negative Indexing:** You can also access elements from the end of the list using negative indices. -1 refers to the last element, -2 to the second last, and so on.
- **Examples:**

Python

```
fruits = ["apple", "banana", "cherry"]
```

`print(fruits[-1])` # Output: cherry

`print(fruits[-2])` # Output: banana

- **Slicing:** You can extract a portion of a list (a sublist) using slicing.
- **Syntax:** `list_name[start:stop:step]`
 - **start:** The starting index (inclusive).
 - **stop:** The ending index (exclusive).
 - **step:** The step size (optional).
- **Examples:**

Python

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
print(numbers[2:5]) # Output: [2, 3, 4]
```

```
print(numbers[:3]) # Output: [0, 1, 2]  
(from beginning to index 3 (exclusive))
```

```
print(numbers[5:]) # Output: [5, 6, 7, 8,  
9] (from index 5 to the end)
```

```
print(numbers[1:8:2]) # Output: [1, 3, 5, 7]  
(every second element from index 1 to 8)
```

```
print(numbers[:]) #Output: [0, 1, 2, 3, 4, 5,  
6, 7, 8, 9] (A copy of the entire list.)
```

```
print(numbers[::-1]) #Output: [9, 8, 7, 6, 5,  
4, 3, 2, 1, 0] (Reversed list)
```

3. Modifying Lists:

- **Changing Elements:** You can change the value of an element by assigning a new value to its index.
- **Examples:**

Python

```
fruits = ["apple", "banana", "cherry"]
```

```
fruits[1] = "grape"
```

print(fruits) # Output: ['apple', 'grape', 'cherry']

- . Adding Elements:**

- **append():** Adds an element to the end of the list.
- **insert():** Inserts an element at a specified index.
- **extend():** Extends a list by appending all the items from an iterable (e.g., another list).

- . Removing Elements:**

- **remove():** Removes the first occurrence of a specified value.
- **pop():** Removes and returns the element at a specified index (or the last element if no index is specified).
- **del:** Deletes an element at a specific index, or deletes the entire list.
- **clear():** Removes all elements from the list.

. Examples

Python

```
fruits = ["apple", "banana"]
```

```
fruits.append("orange")
```

```
print(fruits) #output: ['apple', 'banana',  
'orange']
```

```
fruits.insert(1,"kiwi")
```

```
print(fruits) #output: ['apple', 'kiwi',  
'banana', 'orange']
```

```
more_fruits = ["mango","peach"]
```

```
fruits.extend(more_fruits)
```

```
print(fruits) #output: ['apple', 'kiwi',  
'banana', 'orange', 'mango', 'peach']
```

```
fruits.remove("banana")
```

```
print(fruits) #output: ['apple', 'kiwi',  
'orange', 'mango', 'peach']
```

```
removed_item = fruits.pop(1)
```

```
print(removed_item) #output: kiwi
```

```
print(fruits) #output: ['apple', 'orange',  
'mango', 'peach']
```

```
del fruits[0]
```

```
print(fruits) #output: ['orange', 'mango',  
'peach']
```

```
fruits.clear()
```

```
print(fruits) #output: []
```

Q2. Indexing in lists (positive and negative indexing).

When working with lists (or other sequence types like strings and tuples) in Python, indexing is how you access individual elements. Python offers two main ways to index: positive indexing and negative indexing. Here's a breakdown:

1. Positive Indexing:

- . Concept:**

- Positive indexing starts from the beginning of the list.
- The first element has an index of 0, the second element has an index of 1, and so on.

- . Example:**

Python

```
my_list = ['apple', 'banana', 'cherry', 'date']
```

```
print(my_list[0]) # Output: apple
print(my_list[1]) # Output: banana
print(my_list[2]) # Output: cherry
print(my_list[3]) # Output: date
```

2. Negative Indexing:

- . Concept:**

- Negative indexing starts from the end of the list.
- The last element has an index of -1, the second-to-last element has an index of -2, and so on.

- . Example:**

Python

```
my_list = ['apple', 'banana', 'cherry', 'date']
print(my_list[-1]) # Output: date
print(my_list[-2]) # Output: cherry
print(my_list[-3]) # Output: banana
print(my_list[-4]) # Output: apple
```

Key Points:

- **Zero-Based:** Python uses zero-based indexing, meaning the first element's index is always 0.
- **Use Cases:**
 - Positive indexing is useful when you want to access elements from the beginning of the list.
 - Negative indexing is particularly handy when you need to access elements from the end of the list, especially when you don't know the exact length of the list.
- **Slicing:** Both positive and negative indices can be used when slicing lists, which allows you to extract portions of a list.

Q3. Slicing a list: accessing a range of elements.

List slicing in Python is a powerful feature that allows you to extract a portion of a list, creating a new sublist. Here's a breakdown of how it works:

Basic Syntax:

- **list[start:stop:step]**
 - **start:** The index where the slice begins (inclusive). If omitted, it defaults to 0 (the beginning of the list).
 - **stop:** The index where the slice ends (exclusive). If omitted, it defaults to the end of the list.
 - **step:** The increment between elements in the slice. If omitted, it defaults to 1.

Examples:

Let's use this list as our example:

Python

```
my_list = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- **Getting a sublist:**

Python

```
print(my_list[2:5]) # Output: [2, 3, 4]
```

- This extracts elements from index 2 up to (but not including) index 5.

- **Slicing from the beginning:**

Python

print(my_list[:4]) # Output: [0, 1, 2, 3]

- This extracts elements from the beginning of the list up to (but not including) index 4.

- **Slicing to the end:**

Python

print(my_list[6:]) # Output: [6, 7, 8, 9]

- This extracts elements from index 6 to the end of the list.

- **Using a step:**

Python

print(my_list[1:8:2]) # Output: [1, 3, 5, 7]

- This extracts elements from index 1 to (but not including) index 8, taking every second element.

- **Negative indexing in slicing:**

Python

print(my_list[-3:]) #output: [7,8,9]

print(my_list[:-2]) #output: [0, 1, 2, 3, 4, 5, 6, 7]

`print(my_list[-5:-1])` #output: [5, 6, 7, 8]

- Negative indexes can also be used in slicing.

- **Reversing a list:**

Python

`print(my_list[::-1])` # Output: [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]

- A step of -1 reverses the list.

Key Points:

- Slicing creates a new list. It does not modify the original list.
- If the start index is greater than or equal to the stop index, an empty list is returned.
- Slicing can be very useful for extracting specific portions of data from lists, and manipulating that data.

Lab:

Q1. Write a Python program to create a list with elements of multiple data types (integers, strings, floats, etc.).

```
def create_mixed_list(): """Creates a list
with elements of multiple data types."""
mixed_list = [ 10, "Hello", 3.14, True, [1, 2,
3], # Nested list {"name": "Alice", "age":
30}, # Dictionary (4, 5, 6), # Tuple None,
#None type 'a' #String of length 1 ] return
mixed_list # Example usage my_mixed_list
= create_mixed_list() print(my_mixed_list)
#Demonstrating some of the datatypes
contained within the list. print(f"The
element at index 0 is of type:
{type(my_mixed_list[0])}") print(f"The
element at index 1 is of type:
{type(my_mixed_list[1])}") print(f"The
element at index 2 is of type:
{type(my_mixed_list[2])}") print(f"The
element at index 3 is of type:
{type(my_mixed_list[3])}") print(f"The
element at index 4 is of type:
{type(my_mixed_list[4])}") print(f"The
element at index 5 is of type:
{type(my_mixed_list[5])}") print(f"The
element at index 6 is of type:
{type(my_mixed_list[6])}") print(f"The
element at index 7 is of type:
```

```
{type(my_mixed_list[7])}") print(f"The  
element at index 8 is of type:  
{type(my_mixed_list[8])}")
```

Q2. Write a Python program to access elements at different index positions.

```
def access_elements_by_index(my_list):  
    """ Accesses and prints elements at  
    various index positions in a list. Args:  
    my_list: The list from which to access  
    elements. """ if not my_list: # Check if the  
list is empty print("List is empty.") return  
print("Accessing elements using positive  
indices:") print(f"Element at index 0:  
{my_list[0]}") if len(my_list) > 1:  
print(f"Element at index 1: {my_list[1]}") if  
len(my_list) > 2: print(f"Element at index 2:  
{my_list[2]}") print("\nAccessing elements  
using negative indices:") print(f"Last  
element (index -1): {my_list[-1]}") if  
len(my_list) > 1: print(f"Second last  
element (index -2): {my_list[-2]}") if  
len(my_list) > 2: print(f"Third last element  
(index -3): {my_list[-3]}") # Example Usage  
example_list = [10, 20, 30, 40, 50]
```

```
access_elements_by_index(example_list)
empty_list = []
access_elements_by_index(empty_list)
short_list = [1]
access_elements_by_index(short_list)
short_list2 = [1,2]
access_elements_by_index(short_list2)
```

Practical Examples:

Q1. Write a Python program to create a list of multiple data type elements

```
def create_mixed_data_list(): """Creates a
list with elements of various data
types.""" mixed_list = [ 42, # Integer
"Python", # String 3.14159, # Float True, #
Boolean [1, 2, 3], # Nested list {"name":
"John", "age": 30}, # Dictionary (5, 6), #
Tuple None, # NoneType 'c' #single
character string. ] return mixed_list
```

Q2. Write a Python program to find the length of a list using the len() function.

```
def find_list_length(my_list): """ Finds and
returns the length of a list using the len()
function. Args: my_list: The list whose
length is to be determined. Returns: The
```

```
length of the list (an integer). """  
list_length = len(my_list) return list_length  
# Example Usage my_list1 = [1, 2, 3, 4, 5]  
length1 = find_list_length(my_list1)  
print(f"The length of the list is: {length1}")  
my_list2 = ["apple", "banana", "cherry"]  
length2 = find_list_length(my_list2)  
print(f"The length of the list is: {length2}")  
my_list3 = [] # Empty list length3 =  
find_list_length(my_list3) print(f"The  
length of the list is: {length3}") my_list4 =  
[1, 'a', 3.14, True, [1,2]] length4 =  
find_list_length(my_list4) print(f"The  
length of the list is: {length4}")
```

2. List Operations